

Web System Technologies

IT0049



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Module 3

Routing and Controllers



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Objectives

- To explain the role of routing in a web application.
- To describe how CodeIgniter 4 uses routing to direct HTTP requests to specific controllers and methods.
- To define simple routes in app/Config/Routes.php.
- To implement routes with parameters and understand how to capture and pass them to controller methods.
- To explain the purpose of controllers in the MVC architecture.
- To describe how controllers manage the flow of data between models and views.
- To create a basic controller in CodeIgniter 4.
- To organize controller methods to handle various HTTP requests (GET, POST, etc.).



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Module 3

Routing

What is URI Routing?

Routing is the process of defining how a web application responds to a specific HTTP request made to a particular URI (Uniform Resource Identifier).

In simple terms, routing determines which controller and method are executed when a user visits a specific URL in your application.



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

URI Routing

CodeIgniter has two kinds of routing:

- 1. Defined Route Routing**
- 2. Auto Routing.**

With Defined Route Routing, you can define routes manually. It allows flexible URL.

Auto Routing automatically routes HTTP requests based on conventions and execute the corresponding controller methods. There is no need to define routes manually.

Setting Routing Rules

Routing rules are defined in the **app/Config/Routes.php** file and has the instance of the RouteCollection class (**\$routes**) that permits you to specify your own routing criteria.

Routes can be specified using placeholders or Regular Expressions.



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Setting Routing Rules

When you specify a route, you choose a method to corresponding to HTTP verbs (request method). If you expect a GET request, you use the `get()` method:

```
$routes->get('/', 'Home::index');
```

A route takes the **Route Path** (URI path relative to the BaseURL. /) on the left, and maps it to the **Route Handler** (controller and method **Home::index**) on the right, along with any parameters that should be passed to the controller.



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Setting Routing Rules

The controller and method should be listed by separating the class and its method with a double-colon, like *Users::list*.

```
// Calls $Users->list()  
$routes->get('users', 'Users::list');
```

If that method requires parameters to be passed to it, then they would be listed after the method name, separated by forward-slashes:

```
// Calls $Users->list(1, 23)  
$routes->get('users/1/23', 'Users::list/1/23');
```



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Examples: Basic Routing

A URL containing the word **journals** in the first segment will be mapped to the **\App\Controllers\Blogs** class, and the [default method](#), which is usually **index()**:

```
$routes->get('journals', 'Blogs');
```

A URL containing the segments **blog/joe** will be mapped to the **\App\Controllers\Blogs** class and the **users()** method. The ID will be set to **34**:

```
$routes->get('blog/joe', 'Blogs::users/34');
```



Examples: Basic Routing

A URL with **product** as the first segment, and anything in the second will be mapped to the **\App\Controllers\Catalog** class and the **productLookup()** method:

```
$routes->get('product/:segment', 'Catalog::productLookup');
```

A URL with **product** as the first segment, and a number in the second will be mapped to the **\App\Controllers\Catalog** class and the **productLookupByID()** method passing in the match as a variable to the method:

```
$routes->get('product/:num', 'Catalog::productLookupByID/$1');
```



Examples : HTTP verb routes

You can use any standard HTTP verb (GET, POST, PUT, DELETE, OPTIONS, etc):

```
$routes->post('products', 'Product::feature');
```

```
$routes->put('products/1', 'Product::feature');
```

```
$routes->delete('products/1', 'Product::feature');
```

You can supply multiple verbs that a route should match by passing them in as an array to the **match()** method:

```
$routes->match(['GET', 'PUT'], 'products', 'Product::feature');
```


Specifying Route Path

Placeholders

A typical route might look something like this:

```
$routes->get('product/(:num)', 'Catalog::productLookup');
```

The **first parameter** contains the URI to be matched, while the **second parameter** contains the destination it should be routed to.

In the above example, if the literal word “product” is found in the first segment of the URL path, and a number is found in the second segment, the **Catalog** class and the **productLookup** method are used instead.



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Placeholders

Placeholders are strings that represent a Regular Expression pattern. They are primarily used for readability.

The following placeholders are available for you to use in your routes:

Placeholders	Description
(:any)	will match all characters from that point to the end of the URI. This may include multiple URI segments.
(:segment)	will match any character except for a forward slash (/) restricting the result to a single segment.
(:num)	will match any integer.
(:alpha)	will match any string of alphabetic characters
(:alphanumeric)	will match any string of alphabetic characters or integers, or any combination of the two.
(:hash)	is the same as (:segment), but can be used to easily see which routes use hashed ids.

Reverse Routing

Reverse routing allows you to define the controller and method, as well as any parameters, that a link should go to.

This allows route definitions to change without you having to update your application code.



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Reverse Routing

For example, if you have a route to a photo gallery that you want to link to, you can use the `url_to()` helper function to get the route that should be used.

// The route is defined as:

```
$routes->get('users/(:num)/gallery/(:num)', 'Galleries::showUserGallery/$1/$2');
```

View:

```
<!-- Generate the URI to link to user ID 15, gallery 12: -->
```

```
<a href="<?= url_to('Galleries::showUserGallery', 15, 12) ?>">View Gallery</a>
```

```
<!-- Result: 'http://example.com/users/15/gallery/12' -->
```



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Auto Routing

When no defined route is found that matches the URI, the system will attempt to match that URI against the controllers and methods when Auto Routing is enabled.

Auto Routing can automatically route HTTP requests based on conventions and execute the corresponding controller methods.

Auto Routing is **disabled** by default.



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Auto Routing

Enable Auto Routing

To use it, you need to change the setting \$autoRoute option to true in **app/Config/Routing.php**:

```
public bool $autoRoute = true;
```

And you need to change the property \$autoRoutesImproved to true in **app/Config/Feature.php**:

```
public bool $autoRoutesImproved = true;
```



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Auto Routing

URI SEGMENTS

The segments in the URL, in following with the Model-View-Controller approach, usually represent:

example.com/**class**/**method**/**ID**

- 1.The first segment represents the controller **class** that should be invoked.
- 2.The second segment represents the class **method** that should be called.
- 3.The third, and any additional segments, represent the **ID** and any variables that will be passed to the controller.



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Auto Routing

URI SEGMENTS

Consider this URI:

example.com/index.php/helloworld/hello/1

In the above example, when you send an HTTP request with **GET** method, Auto Routing would attempt to find a controller named **App\Controllers\Helloworld** and executes **getHello()** method with passing '1' as the first argument.



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Module 3

Controller

Controller

Controller is the heart of your application.

What is a Controller?

A Controller is simply a class file that handles an HTTP request. [URI Routing](#) associates a URI with a controller. It returns a view string or Response object.

Every controller you create should extend BaseController class. This class provides several features that are available to all of your controllers.

Controller

Constructor

CodeIgniter's Controller has a special constructor `initController()`. It will be called by the framework after PHP's constructor `__construct()` execution.

If you want to override the `initController()`, don't forget to add `parent::initController($request, $response, $logger);`

```
<?php

namespace App\Controllers;

use CodeIgniter\HTTP\RequestInterface;
use CodeIgniter\HTTP\ResponseInterface;
use Psr\Log\LoggerInterface;

class Product extends BaseController
{
    public function initController(
        RequestInterface $request,
        ResponseInterface $response,
        LoggerInterface $logger
    ) {
        parent::initController($request, $response, $logger);

        // Add your code here.
    }

    // ...
}
```



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

initController() Method

The `initController()` method sets the following three properties.

1. Request Object - The application's main [Request Instance](#) is always available as a class property, `$this->request`.
2. Response Object - The application's main [Response Instance](#) is always available as a class property, `$this->response`.
3. Logger Object - An instance of the [Logger](#) class is available as a class property, `$this->logger`.



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Helpers

You can define an array of helper files as a class property. Whenever the controller is loaded these helper files will be automatically loaded into memory so that you can use their methods anywhere inside the controller:

```
<?php

namespace App\Controllers;

class MyController extends BaseController
{
    protected $helpers = ['url', 'form'];
}
```



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

forceHTTPS

A convenience method for forcing a method to be accessed via HTTPS is available within all controllers:

```
<?php  
  
if (! $this->request->isSecure()) {  
    $this->forceHTTPS();  
}
```



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Validating Data

`$this->validateData()`

To simplify data checking, the controller also provides the convenience method `validateData()`.

The method accepts:

- (1) an array of data to validate,
- (2) an array of rules,
- (3) an optional array of custom error messages to display if the items are not valid,
- (4) an optional database group to use.

```
<?php

namespace App\Controllers;

class StoreController extends BaseController
{
    public function product(int $id)
    {
        $data = [
            'id' => $id,
            'name' => $this->request->getPost('name'),
        ];

        $rule = [
            'id' => 'integer',
            'name' => 'required|max_length[255]',
        ];

        if (! $this->validateData($data, $rule)) {
            return view('store/product', [
                'errors' => $this->validator->getErrors(),
            ]);
        }

        // ...
    }
}
```



Let's try it: Hello World!

Using your text editor, create a file called **Helloworld.php**, and put the following code in it. You will notice that the **Helloworld** Controller is extending the **BaseController**.

*Note: You can also extend the **CodeIgniter\Controller** if you do not need the functionality of the **BaseController**.*

Then save the file to your **app/Controllers** directory.

```
<?php

namespace App\Controllers;

class Helloworld extends BaseController
{
    public function getIndex()
    {
        return 'Hello World!';
    }
}
```



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Let's try it: Hello World!

Now visit your site using a URL similar to this:

example.com/index.php/helloworld

If you did it right you should see:

Hello World!



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Examples

This is valid:

```
<?php

namespace App\Controllers;

class Helloworld extends BaseController
{
    // ...
}
```

Note : Class filename is Helloworld.php

These are not:

```
<?php

namespace App\Controllers;

class helloworld extends BaseController
{
    // ...
}
```

```
<?php

namespace App\Controllers;

class HelloWorld extends BaseController
{
    // ...
}
```



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Methods

Method Visibility

When you define a method that is executable via HTTP request, the method must be declared as **public**.

Default Method

The method (HTTP verb + **Index()**) is called the **default method**, and loaded if the **second segment** of the URI is empty.

Normal Methods

The second segment of the URI determines which method in the controller gets called.

Now load the following URL to see the `getComment()` method:
`example.com/index.php/helloworld/comment/`

```
<?php

namespace App\Controllers;

class Helloworld extends BaseController
{
    public function getIndex()
    {
        return 'Hello World!';
    }

    public function getComment()
    {
        return 'I am not flat!';
    }
}
```



Passing URI Segments to Your Methods

If your URI contains more than two segments they will be passed to your method as parameters.

For example, let's say you have a URI like this:

example.com/index.php/products/shoes/sandals/123

Your method will be passed URI segments 3 and 4 ('sandals' and '123'):

```
<?php

namespace App\Controllers;

class Products extends BaseController
{
    public function getShoes($sandals, $id)
    {
        return $sandals . $id;
    }
}
```

Default Controller

The Default Controller is a special controller that is used when a URI ends with a directory name or when a URI is not present, as will be the case when only your site root URL is requested.



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

Defining Default Controller

Let's try it with the Helloworld controller.

To specify a default controller open your **app/Config/Router.php** file and set this property:

```
public string $defaultController = 'Helloworld';
```

And comment out the line in **app/Config/Routes.php**:

```
$routes->get('/', 'Home::index');
```

If you now browse to your site without specifying any URI segments you'll see the "Hello World" message.



FEU ALABANG



FEU DILIMAN



FEU TECH

Technology Driven by Innovation

